

Ejercicio de diseño: empresa de alquiler de vehículos

a) Elección de un diseño para esta realidad

Tipos Abstractos de Datos elegidos

Vehículos = Secuencia (Vehículo)

Vehículo = matrícula × marca × año

Auto = cant_puertas × precio_base × Vehículo

Moto = cilindrada × precio_base × Vehículo

Tipos de datos

- matrícula: String
- marca: String
- año: Integer
- cant_puertas: Integer
- cilindrada: Integer
- precio_base: Real

Observaciones

- Vehículo contiene los datos comunes a autos y motos: matrícula, marca y año.
- Auto y Moto heredan de Vehículo.
- Se utiliza una secuencia porque el sistema debe almacenar varios vehículos.
- La matrícula identifica de manera única a cada vehículo, por lo que no debe repetirse.
- Vehículo es una clase abstracta, ya que los objetos concretos serán autos o motos.
- El precio de alquiler diario se calcula de forma diferente según el tipo de vehículo.

- En Auto, el precio se calcula mediante:

precio_base + cant_puertas × 100

- En Moto, el precio se calcula mediante:

precio_base + cilindrada × 0,5

- El método precioAlquilerDiario() es polimórfico, porque posee una implementación diferente en Auto y en Moto.
- SecuenciaVehiculos almacena y administra la colección de vehículos.
- FachadaLogica centraliza las operaciones solicitadas por el sistema.

b) Desglose de requerimientos

Requerimiento 1a: Alta de vehículo

Entrada:
Un vehículo, que puede ser un auto o una moto.

Salida:
Confirmación de alta o mensaje de error.

Proceso:
Se recibe el vehículo y se busca en la secuencia otro vehículo con la misma matrícula. Si la matrícula ya existe, no se realiza el alta. Si no existe, el vehículo se agrega a la secuencia y se informa que la operación fue realizada correctamente.

Requerimiento 1b: Baja de vehículo

Entrada:
Matrícula del vehículo que se desea eliminar.

Salida:
Confirmación de baja o mensaje de error.

Proceso:
Se busca en la secuencia un vehículo con la matrícula recibida. Si no existe, se informa que la matrícula no está registrada. Si existe, el vehículo se elimina de la secuencia.

Requerimiento 2: Calcular la suma de precios de alquiler diario

Entrada:
La secuencia de vehículos.

Salida:
La suma total de los precios de alquiler diario.

Proceso:
Se inicializa un acumulador en cero. Luego se recorren todos los vehículos de la secuencia. Para cada vehículo se invoca el método precioAlquilerDiario() y el resultado se suma al acumulador. Finalmente, se retorna el total.

Si la secuencia está vacía, el resultado es cero.

Requerimiento 3: Obtener los vehículos con mayor y menor precio de alquiler diario

Entrada:
La secuencia de vehículos.

Salida:
Los datos del vehículo con mayor precio de alquiler diario y los datos del vehículo con menor precio de alquiler diario.

Proceso:
Se verifica que la secuencia no esté vacía. Se toma el primer vehículo como mayor y menor. Luego se recorren los demás vehículos y se comparan sus precios de alquiler diario. Si un vehículo posee un precio mayor, se actualiza el vehículo mayor. Si posee un precio menor, se actualiza el vehículo menor. Finalmente, se retornan ambos vehículos.

Si existe un solo vehículo, será al mismo tiempo el de mayor y menor precio.

Requerimiento 4: Contar motos por cilindrada

Entrada:
Una cilindrada.

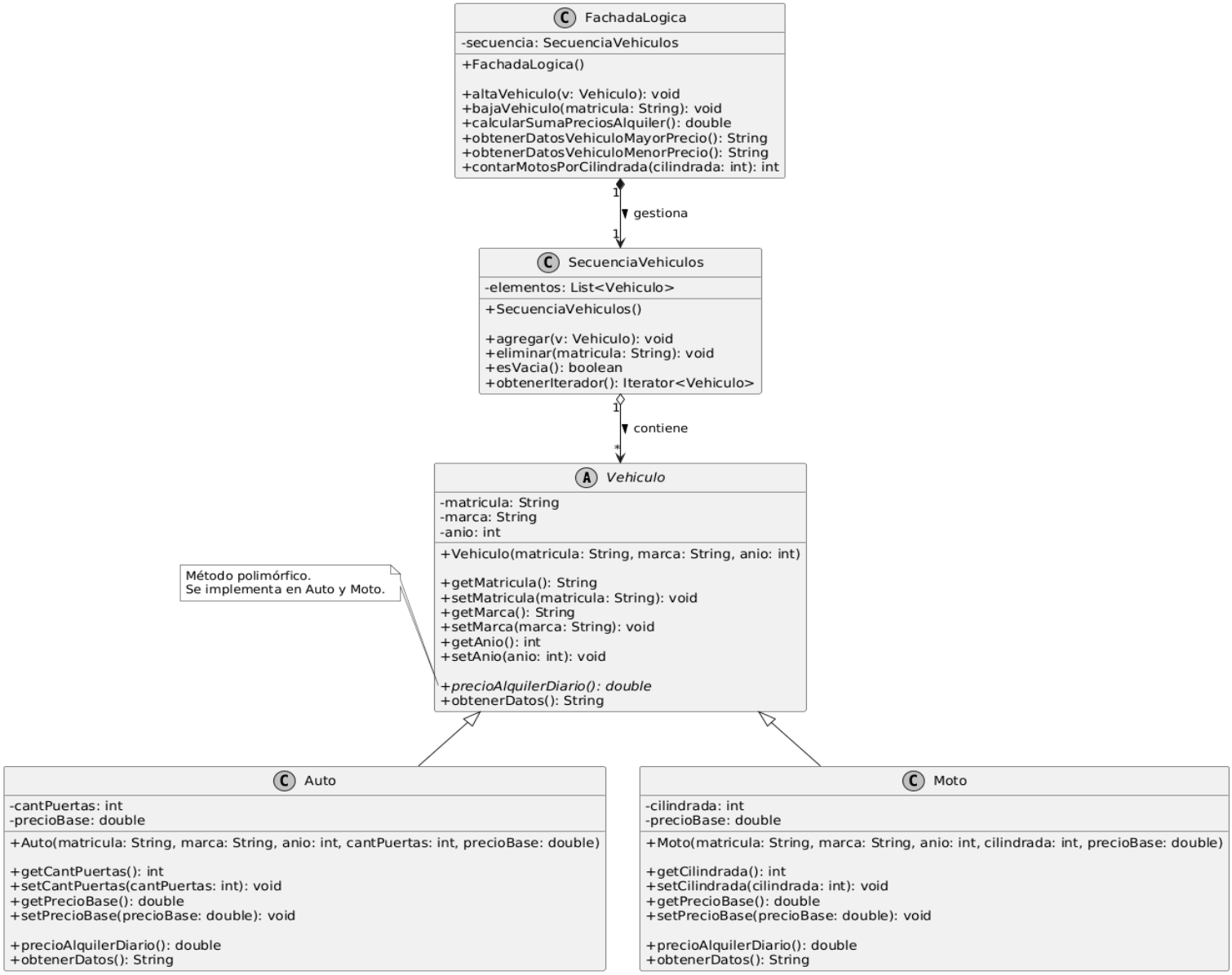
Salida:
Cantidad de motos que poseen la cilindrada indicada.

Proceso:
Se inicializa un contador en cero. Luego se recorren todos los vehículos de la secuencia. Para cada vehículo se verifica si es una moto. Si lo es, se compara su

cilindrada con la cilindrada recibida. Si coinciden, se incrementa el contador. Finalmente, se retorna la cantidad obtenida.

c) Especificación del diseño elegido

Diagrama de clases de implementación



Clase Vehiculo

La clase Vehiculo es abstracta.

Atributos:

- matricula: String
- marca: String
- anio: int

Constructor:

- Vehiculo(matricula: String, marca: String, anio: int)

Selectores:

- getMatricula(): String
- getMarca(): String
- getAnio(): int

Modificadores:

- setMatricula(matricula: String): void
- setMarca(marca: String): void
- setAnio(anio: int): void

Métodos:

- precioAlquilerDiario(): double
- obtenerDatos(): String

El método precioAlquilerDiario() es abstracto y se implementa de forma diferente en Auto y Moto.

Clase Auto

La clase Auto hereda de Vehiculo.

Atributos:

- cantPuertas: int
- precioBase: double

Constructor:

- Auto(matricula: String, marca: String, anio: int, cantPuertas: int, precioBase: double)

Selectores:

- getCantPuertas(): int
- getPrecioBase(): double

Modificadores:

- setCantPuertas(cantPuertas: int): void
- setPrecioBase(precioBase: double): void

Métodos:

- precioAlquilerDiario(): double
- obtenerDatos(): String

El precio de alquiler diario se calcula mediante:

precioBase + cantPuertas × 100

Clase Moto

La clase Moto hereda de Vehiculo.

Atributos:

- cilindrada: int
- precioBase: double

Constructor:

- Moto(matricula: String, marca: String, anio: int, cilindrada: int, precioBase: double)

Selectores:

- getCilindrada(): int
- getPrecioBase(): double

Modificadores:

- setCilindrada(cilindrada: int): void

- setPrecioBase(precioBase: double): void

Métodos:

- precioAlquilerDiario(): double
- obtenerDatos(): String

El precio de alquiler diario se calcula mediante:

precioBase + cilindrada × 0,5

Clase SecuenciaVehiculos

Atributos:

- elementos: List

Constructor:

- SecuenciaVehiculos()

Métodos:

- agregar(v: Vehiculo): void
- eliminar(matricula: String): void
- esVacia(): boolean
- obtenerIterador(): Iterator

La clase SecuenciaVehiculos almacena y permite recorrer la colección de vehículos.

Clase FachadaLogica

Atributos:

- secuencia: SecuenciaVehiculos

Constructor:

- FachadaLogica()

Métodos:

- altaVehiculo(v: Vehiculo): void
- bajaVehiculo(matricula: String): void

- calcularSumaPreciosAlquiler(): double
- obtenerDatosVehiculoMayorPrecio(): String
- obtenerDatosVehiculoMenorPrecio(): String
- contarMotosPorCilindrada(cilindrada: int): int

La clase FachadaLogica centraliza las operaciones principales del sistema: altas, bajas, suma de precios, búsqueda del mayor y menor precio y conteo de motos según cilindrada.